

[► View Source](#)

```
def rot_axis_from_rot_mat(rot_mat):
```

[► View Source](#)

Gibt die Rotationsachse von rot_mat zurueck.

Parameters

- **rot_mat**: Die Rotationsmatrix als mehrdimensionales Array.

Returns

Rotationsachse als normalisierter Vektor z.B [x,y,z].

```
def quaternion_to_euler(x, y, z, w, order='ZYZ'):
```

[► View Source](#)

Wandelt ein Quaternion in Eulerwinkel um. Dabei wird die Übergebene Rotationsreihenfolge für die Eulerwinkel beachtet.

Parameters

- **x**: x-Komponente des Quaternions.
- **y**: y-Komponente des Quaternions.
- **z**: z-Komponente des Quaternions.
- **w**: w-Komponente des Quaternions.
- **order**: Rotationsreihenfolge für die Eulerwinkel als String.

Returns

Die Eulerwinkel als Array in der Reihenfolge, wie es order vorgibt, z.B order="ZXY" rückgabe-> [z,x,y].

```
def euler_to_rot_mat(angles, order='ZYZ'):
```

[► View Source](#)

Wandelt Eulerwinkel in eine Rotationsmatrix um.

Parameters

- **angles**: Die Eulerwinkel in Grad. Diese Müssen in der Reihenfolge angegeben werden wie es order vorgibt z.B angles=[y,x,z] order="YXZ".
- **order**: Rotationsreihenfolge für die Eulerwinkel als String.

Returns

Die Rotationsmatrix als mehrdimensionales Array.

```
def rot_matrix_to_euler(rot_mat, order='ZYZ'):
```

[► View Source](#)

Wandelt eine Rotationsmatrix in Eulerwinkel um.

Parameters

- **rot_mat**: Die Rotationsmatrix.
- **order**: Rotationsreihenfolge für die Eulerwinkel als String.

Returns

Die Eulerwinkel. Diese werden in der Reihenfolge zurueckgegeben, wie es order vorgibt z.B order="ZXY" rueckgabe-> [Z,X,Y].

def rot_matrix_to_quaternion(rot_mat):

► [View Source](#)

Wandelt eine Rotationsmatrix in ein Quaternion um.

Parameters

- **rot_mat**: Die Rotationsmatrix als mehrdimensionales Array.

Returns

das Quaternion als Array.

def euler_to_quaternion(angles, order='YZZ'):

► [View Source](#)

Wandelt Eulerwinkel in ein Quaternion um.

Parameters

- **angles**: Die Eulerwinkel in Grad. Diese müssen in der Reihenfolge angegeben werden, wie es order vorgibt z.B angles=[y,x,z] order="YXZ".
- **order**: Rotationsreihenfolge für die Eulerwinkel als String.

Returns

Das Quaternion als Array.

def compute_normals(vertices, indices):

► [View Source](#)

Generiert die Normalen fuer alle Dreiecke die sich aus vertices und indices ergeben.

Parameters

- **vertices**: Die Punkte im Raum aus denen die Dreiecke bestehen, für die die Normalen berechnet werden als Array.
- **indices**: Die Indexe zum Verbinden der Punkte zu Dreiecken, für welche dann die Normalen berechnet werden als Array.

Returns

Ein Array, welches die Normalen enthaelt.

def order_angles(x, y, z, order):

► [View Source](#)

Ordnet die uebergebenen Eulerwinkel nach order und gibt diese als Array zurueck.

Parameters

- **x**: Eulerwinkel um x.
- **y**: Eulerwinkel um y.
- **z**: Eulerwinkel um z.

Returns

Die geordneten Eulerwinkel als Array.

def quaternion_multiply(q1, q2):

► [View Source](#)

Multipliziert zwei Quaternions.

Parameters

- **q1**: Das erste Quaternion als Liste oder Array [x, y, z, w].
- **q2**: Das zweite Quaternion als Liste oder Array [x, y, z, w].

Returns

Das Ergebnis der Quaternion-Multiplikation als Liste [x, y, z, w].

def create_axes(len, font_scale=0.4, show_labels=**True**, name=""):

► [View Source](#)

Erstellt ein 3D-Koordinatensystem mit den Achsen X, Y, Z und optionalen Beschriftungen.

Parameters

- **len**: Länge der Achsen.
- **font_scale**: Skalierung der Schriftgröße für die Achsenbeschriftungen.
- **show_labels**: Boolescher Wert, der angibt, ob die Achsenbeschriftungen angezeigt werden sollen.
- **name**: Optionaler Name, der in der Mitte des Koordinatensystems angezeigt wird.

Returns

Ein 3D-Objekt (Group), das das Koordinatensystem mit Achsen und optionalen Beschriftungen enthält.

def create_grid_XY(size, density):

► [View Source](#)

Erstellt ein 3D-Gitter im XY-Plane mit der angegebenen Größe und Dichte.

Parameters

- **size**: Die Größe des Gitters (die Ausdehnung in X und Y Richtung).
- **density**: Die Dichte des Gitters, die angibt, wie viele Linien innerhalb des Gitters erstellt werden.

Returns

Ein 3D-Objekt (Group), das das Gitter mit Linien im XY-Plane enthält.

def create_grid_XZ(size, density):

[► View Source](#)

Erstellt ein 3D-Gitter im XZ-Plane mit der angegebenen Größe und Dichte.

Parameters

- **size**: Die Größe des Gitters (die Ausdehnung in X und Y Richtung).
- **density**: Die Dichte des Gitters, die angibt, wie viele Linien innerhalb des Gitters erstellt werden.

Returns

Ein 3D-Objekt (Group), das das Gitter mit Linien im XZ-Plane enthält.

def apply_rot_matrix(mesh, rot_mat):

[► View Source](#)

Wendet eine Rotationsmatrix auf ein Mesh-Objekt an, indem die Matrix in ein Quaternion umgewandelt wird und auf das bestehende Quaternion des Meshs angewendet wird.

Parameters

- **mesh**: Das Mesh-Objekt, auf das die Rotation angewendet werden soll. Erwartet wird, dass das Mesh ein `quaternion`-Attribut besitzt.
- **rot_mat**: Die Rotationsmatrix, die auf das Mesh angewendet werden soll. Muss eine 3x3 Matrix sein.

def createQuad(pos, width, height, depth, color=[0, 255, 0], transparent=**True**): [► View Source](#)

Erzeugt ein Quader-Mesh (Box) mit der angegebenen Position, Größe und Farbe.

Parameters

- **pos**: Die Position des Quaders als Array oder Tuple [x, y, z].
- **width**: Die Breite des Quaders.
- **height**: Die Höhe des Quaders.
- **depth**: Die Tiefe des Quaders.
- **color**: Die Farbe des Quaders als Array [R, G, B], wobei jede Komponente im Bereich 0-255 liegt. Standardmäßig grün [0, 255, 0].
- **transparent**: Ein Boolean-Wert, der angibt, ob das Material transparent sein soll. Standardmäßig `True`.

Returns

Ein Mesh-Objekt, das den Quader darstellt, mit der angegebenen Position, Größe und Farbe.

```
def create_cylinder(
```

```
    pos,  
    radiusTop=1,  
    radiusBottom=1,  
    height=2,  
    radialSegments=32,  
    color=[255, 0, 0],  
    transparent=True
```

```
):
```

► [View Source](#)

Erstellt ein Zylinder-Mesh mit der angegebenen Position, Größe und Farbe.

Parameters

- **pos**: Die Position des Zylinders als Array oder Tuple [x, y, z].
- **radiusTop**: Der Radius des Zylinders an der Oberseite. Standardwert ist 1.
- **radiusBottom**: Der Radius des Zylinders an der Unterseite. Standardwert ist 1.
- **height**: Die Höhe des Zylinders. Standardwert ist 2.
- **radialSegments**: Die Anzahl der radialen Segmente des Zylinders, die die Auflösung rund um den Zylinder bestimmen. Standardwert ist 32.
- **color**: Die Farbe des Zylinders als Array [R, G, B], wobei jede Komponente im Bereich 0-255 liegt. Standardwert ist [255, 0, 0] (Rot).
- **transparent**: Ein Boolean-Wert, der angibt, ob das Material transparent sein soll. Standardmäßig `True`.

Returns

Ein Mesh-Objekt, das den Zylinder darstellt, mit der angegebenen Position, Größe und Farbe.

```
def apply_rot_matrix_animated(mesh, rot_mat, speed=100, show_rot_axis=True):
```

► [View Source](#)

Wendet eine Rotationsmatrix auf ein Mesh an und rotiert es animiert mit einer gegebenen Geschwindigkeit. Dabei kann optional eine Rotationsachse angezeigt werden.

Parameters

- **mesh**: Das Mesh, das rotiert werden soll.
- **rot_mat**: Die Rotationsmatrix, die auf das Mesh angewendet werden soll.
- **speed**: Die Geschwindigkeit der Animation, angegeben als Anzahl der Frames pro Sekunde. Standardwert ist 100.
- **show_rot_axis**: Ein Boolean-Wert, der angibt, ob die Rotationsachse angezeigt werden soll. Standardwert ist `True`.

```
def slerp_quaternion(q1, q2, t):
```

► [View Source](#)

Führt eine Spherical Linear Interpolation (SLERP) zwischen zwei Quaternionen durch. Interpoliert die Rotation zwischen q_1 und q_2 basierend auf dem Interpolationswert t .

Parameters

- **q1**: Das erste Quaternion, das die Anfangsrotation beschreibt.
- **q2**: Das zweite Quaternion, das die Endrotation beschreibt.
- **t**: Der Interpolationswert, der zwischen 0 und 1 liegen muss. Ein Wert von 0 entspricht der Rotation von q_1 und ein Wert von 1 entspricht der Rotation von q_2 .

Returns

Das interpolierte Quaternion, das die Rotation zwischen q_1 und q_2 bei dem gegebenen Wert von t beschreibt.

Raises

- **ValueError**: Wenn der Interpolationswert t nicht zwischen 0 und 1 liegt.

def line_wheel_driven_robot(dummy, vel, steps):

► [View Source](#)

Simuliert die Bewegung eines radgetriebenen Roboters entlang einer Linie und erzeugt dabei Liniensegmente zur Visualisierung des Pfades.

Bei jedem Schritt wird das Dummy-Objekt gemäß der gegebenen Geschwindigkeit bewegt. In regelmäßigen Abständen (alle 64 Schritte) wird ein Liniensegment vom Startpunkt dieses Abschnitts zum aktuellen Punkt erstellt, um die Trajektorie sichtbar zu machen.

Parameters

- **dummy**: Ein Mesh-Objekt, das die Position und Orientierung des Roboters repräsentiert.
- **vel**: Ein Geschwindigkeitsvektor $[v_x, v_y, \omega_z]$, bestehend aus Translation in lokaler X/Y-Richtung und Rotation um Z.
- **steps**: Anzahl der Bewegungs-Iterationen (Zeitschritte).

Returns

Eine `three.Group`, die alle erzeugten Liniensegmente enthält (als Trajektorie).

def set_scale(mesh, scale):

► [View Source](#)

Setzt die Skalierung eines Mesh-Objekts.

Parameters

- **mesh**: Das Mesh-Objekt, dessen Skalierung angepasst werden soll.
- **scale**: Ein Array, das den Skalierungsfaktor für jede Achse (x, y, z) angibt, z.B. $[1, 2, 1]$.

def set_scale_animated(mesh, scale):

► [View Source](#)

Setzt die Skalierung eines Mesh-Objekts animiert, indem es schrittweise die Größe verändert.

Parameters

- **mesh**: Das Mesh-Objekt, dessen Skalierung angepasst werden soll.
- **scale**: Ein Array, das die Ziel-Skalierungswerte für jede Achse (x, y, z) angibt, z.B. [1, 2, 1].

def rotate_animated(mesh, angles, order='ZYZ'):

► [View Source](#)

Führt eine animierte lokale Rotation eines Mesh-Objekts durch. Die Rotation erfolgt achsweise entsprechend der angegebenen Reihenfolge.

Beispiel: order="YXZ" → angles=[Winkel um Y, Winkel um X, Winkel um Z]

Während der Animation wird die Rotation in drei Schritten durchgeführt – einer pro Achse – und am Ende exakt auf das Ziel-Quaternion gesetzt, um numerische Fehler auszugleichen.

Parameters

- **mesh**: Das 3D-Mesh-Objekt, das rotiert werden soll.
- **angles**: Eine Liste mit Rotationswinkeln (in Grad), die in der Reihenfolge `order` angegeben sind.
- **order**: Die Rotationsreihenfolge (z. B. "ZYZ", "YXZ", etc.)

def rotate_global_animated(mesh, angles, order='ZYZ'):

► [View Source](#)

Führt eine animierte globale Rotation eines Mesh-Objekts durch. Die Drehung erfolgt achsweise gemäß der angegebenen Rotationsreihenfolge (z. B. "ZYZ"). Die Winkel in `angles` müssen in der **Reihenfolge der order -Zeichen** angegeben werden.

Beispiel: Bei order="YXZ" → angles=[Winkel um Y, Winkel um X, Winkel um Z]

Die Funktion führt die Rotation in drei separaten animierten Phasen durch – jeweils eine für jede Achse in `order`.

Parameters

- **mesh**: Das 3D-Objekt (Mesh), das rotiert werden soll.
- **angles**: Eine Liste von Rotationswinkeln (in Grad), entsprechend der Reihenfolge in `order`.
- **order**: Die Rotationsreihenfolge als String, z. B. "ZYZ", "YXZ", etc.

def move(robot, vel, steps=1):

► [View Source](#)

Bewegt ein Roboterobjekt in mehreren Schritten entsprechend der gegebenen Geschwindigkeit und Rotation.

Die Funktion kombiniert Translation und Rotation:

- Zuerst wird eine Rotation um die Z-Achse angewendet, basierend auf dem dritten Element des Geschwindigkeitsvektors `vel[2]`.

- Anschließend wird eine Translation basierend auf der aktuellen Ausrichtung (Z-Rotation) des Roboters ausgeführt.
- Die Bewegung wird für eine angegebene Anzahl von Schritten (`steps`) wiederholt.
- Nach jeweils 4 Schritten erfolgt eine kurze Pause zur visuellen Glättung.

Parameters

- **robot**: Das Objekt (z. B. ein Mesh), das bewegt werden soll. Es muss `quaternion` und `position` Attribute besitzen.
- **vel**: Ein Geschwindigkeitsvektor `[v_x, v_y, w_z]` , wobei `v_x` und `v_y` die Translation in der lokalen X- und Y-Richtung und `w_z` die Rotation um die Z-Achse ist (in Radiant).
- **steps**: Die Anzahl der Schritte, die die Bewegung ausführen soll (Standard: 1).

Returns

Die finale Position des Roboters nach der Bewegung.

def rotate_global(mesh, angles, order='ZYZ'):

► [View Source](#)

Führt eine globale Rotation eines Mesh-Objekts durch, basierend auf den übergebenen Eulerwinkeln und einer Rotationsreihenfolge.

Parameters

- **mesh**: Das Mesh-Objekt, das rotiert werden soll.
- **angles**: Die Eulerwinkel in Grad, die die Rotation definieren. Die Reihenfolge muss dem angegebenen "order"-Parameter entsprechen.
- **order**: Die Rotationsreihenfolge als String (z.B. "ZYZ", "XYZ", etc.). Standardmäßig "ZYZ".

def rotate(mesh, angles, order='ZYZ'):

► [View Source](#)

Führt eine Rotation eines Mesh-Objekts basierend auf den übergebenen Eulerwinkeln und einer Rotationsreihenfolge durch.

Parameters

- **mesh**: Das Mesh-Objekt, das rotiert werden soll.
- **angles**: Die Eulerwinkel in Grad, die die Rotation definieren. Die Reihenfolge muss dem angegebenen "order"-Parameter entsprechen.
- **order**: Die Rotationsreihenfolge als String (z.B. "ZYZ", "XYZ", etc.). Standardmäßig "ZYZ".

Returns

Keine Rückgabe. Das Mesh wird direkt rotiert.

def set_rotation(mesh, angles, order='ZYZ'):

► [View Source](#)

Setzt die Rotation eines Mesh-Objekts auf die übergebenen Eulerwinkel und die Rotationsreihenfolge.

Parameters

- **mesh**: Das Mesh-Objekt, dessen Rotation gesetzt werden soll.
- **angles**: Die Eulerwinkel in Grad, die die gewünschte Rotation definieren. Die Reihenfolge muss dem angegebenen "order"-Parameter entsprechen.
- **order**: Die Rotationsreihenfolge als String (z.B. "ZYZ", "XYZ", etc.). Standardmäßig "ZYZ".

def set_rotation_global(mesh, angles, order='ZYZ'):

► [View Source](#)

Setzt die globale Rotation eines Mesh-Objekts auf die übergebenen Eulerwinkel und die Rotationsreihenfolge.

Parameters

- **mesh**: Das Mesh-Objekt, dessen globale Rotation gesetzt werden soll.
- **angles**: Die Eulerwinkel in Grad, die die gewünschte Rotation definieren. Die Reihenfolge muss dem angegebenen "order"-Parameter entsprechen.
- **order**: Die Rotationsreihenfolge als String (z.B. "ZYZ", "XYZ", etc.). Standardmäßig "ZYZ".

def translate(mesh, vec):

► [View Source](#)

Verschiebt ein Mesh-Objekt um einen gegebenen Vektor in den drei Raumachsen.

Parameters

- **mesh**: Das Mesh-Objekt, das verschoben werden soll.
- **vec**: Der Verschiebungsvektor als Array oder Liste [x, y, z], der die Verschiebung in den jeweiligen Raumachsen angibt.

def set_translation(mesh, vec):

► [View Source](#)

Setzt die Position eines Mesh-Objekts auf die angegebenen Koordinaten.

Parameters

- **mesh**: Das Mesh-Objekt, dessen Position gesetzt werden soll.
- **vec**: Der Ziel-Vektor als Array oder Liste [x, y, z], der die neue Position des Meshs im Raum angibt.

def set_translation_animated(mesh, vec, speed=50.0):

► [View Source](#)

Bewegt die Position eines Mesh-Objekts animiert von der aktuellen Position zu einer angegebenen Zielposition.

Parameters

- **mesh**: Das Mesh-Objekt, dessen Position animiert geändert werden soll.
- **vec**: Der Ziel-Vektor als Array oder Liste [x, y, z], zu dem die Position des Meshs bewegt werden soll.

- **speed:** Die Geschwindigkeit der Animation. Ein höherer Wert bedeutet eine schnellere Bewegung.

def translate_animated(mesh, vec, speed=50.0):

► [View Source](#)

Bewegt die Position eines Mesh-Objekts animiert um einen angegebenen Vektor von der aktuellen Position.

Parameters

- **mesh:** Das Mesh-Objekt, dessen Position animiert geändert werden soll.
- **vec:** Der Verschiebungs-Vektor als Array oder Liste [dx, dy, dz], um den die Position des Meshs verändert werden soll.
- **speed:** Die Geschwindigkeit der Animation. Ein höherer Wert bedeutet eine schnellere Bewegung.

class Environment:

► [View Source](#)

Eine 3D-Umgebung, die eine Szene mit Kamera, Lichtquellen, Achsen, Gittern und Widgets für die Interaktivität erstellt.

Diese Klasse erstellt eine 3D-Umgebung mit einer Vielzahl von Features, darunter eine Kamera, Lichtquellen, Achsen- und Gitterdarstellung sowie Steuerungen zur Anpassung von Objekten in der Szene (z.B. Rotation, Skalierung, Translation).

Parameters

- **width: Die Breite der Ansicht in Pixeln (Standard: 700).**
- **height: Die Höhe der Ansicht in Pixeln (Standard: 500).**
- **frame: Ein 3D-Achsenobjekt, das als Referenzrahmen in der Szene hinzugefügt wird (Standard: create_axes(8, name="B")).**
- **grid: Ein Gitterobjekt, das in der Szene angezeigt wird (Standard: create_grid_XY(14, 0.5)).**
- **up: Die Richtung der "Oben"-Achse, die die Orientierung der Kamera bestimmt (Standard: [0, 0, 1]).**

Diese Klasse enthält Methoden, um:

- die Sichtbarkeit von Gitter und Achsen zu steuern,
- interaktive Widgets für Objekte zu erstellen (Translation, Rotation, Skalierung),
- Objekte der Szene hinzuzufügen,
- globale oder lokale Transformationen auf Objekte anzuwenden.

Weitere Features:

- Die Umgebung kann mit einer interaktiven Steuerung für Kamera und Objekte angezeigt werden.

- Widgets für die Manipulation von Objekten (Translation, Rotation, Skalierung) können zur Szene hinzugefügt werden.

Environment(width=700, height=500, frame=**None**, grid=**None**, up=[0, 0, 1])

► [View Source](#)

Initialisiert eine neue 3D-Umgebung.

Parameters

- **width**: Die Breite der Ansicht.
- **height**: Die Höhe der Ansicht.
- **frame**: Ein 3D-Achsenobjekt, das in der Szene hinzugefügt wird.
- **grid**: Ein Gitterobjekt, das in der Szene angezeigt wird.
- **up**: Die Richtung der "Oben"-Achse für die Kamera.

frame

grid

scene

camera

light

renderer

frame_widgets

widgets

def toggle_grid(self, change):

► [View Source](#)

Schaltet die Sichtbarkeit des Gitters um.

Parameters

- **change**: Das Ereignis, das diese Funktion auslöst (wird nicht genutzt).

def toggle_axes(self, change):

► [View Source](#)

Schaltet die Sichtbarkeit der Achsen um.

Parameters

- **change**: Das Ereignis, das diese Funktion auslöst (wird nicht genutzt).

def set_frame_widgets(self, bool):

► [View Source](#)

Aktiviert oder deaktiviert die Anzeige von Frame-Widgets.

Parameters

- **bool**: Wenn True, werden die Widgets angezeigt, andernfalls ausgeblendet.

def add(self, objekts):

► [View Source](#)

Fügt ein oder mehrere Objekte zur Szene hinzu.

Parameters

- **objekts**: Ein einzelnes Objekt oder eine Liste von Objekten, die zur Szene hinzugefügt werden.

def add_widget(self, widget):

► [View Source](#)

Fügt ein Widget zur Umgebung hinzu. Dabei kann es sich auch um ein Buendel von Widgets in einer HBox oder einer VBox handeln.

Parameters

- **widget**: Das Widget, das der Umgebung hinzugefügt werden soll.

def add_gizmo_controls(self, obj, translation=**True**, rotation=**True**, scale=**True**):

► [View Source](#)

Fügt ein Gizmo-Steurelement zur Manipulation eines Objekts in der Umgebung hinzu (Translation, Rotation, Skalierung).

Parameters

- **obj**: Das Objekt, das manipuliert werden soll.
- **translation**: Wenn True, werden Schieberegler für die Translation angezeigt.
- **rotation**: Wenn True, werden Schieberegler für die Rotation angezeigt.
- **scale**: Wenn True, werden Schieberegler für die Skalierung angezeigt.